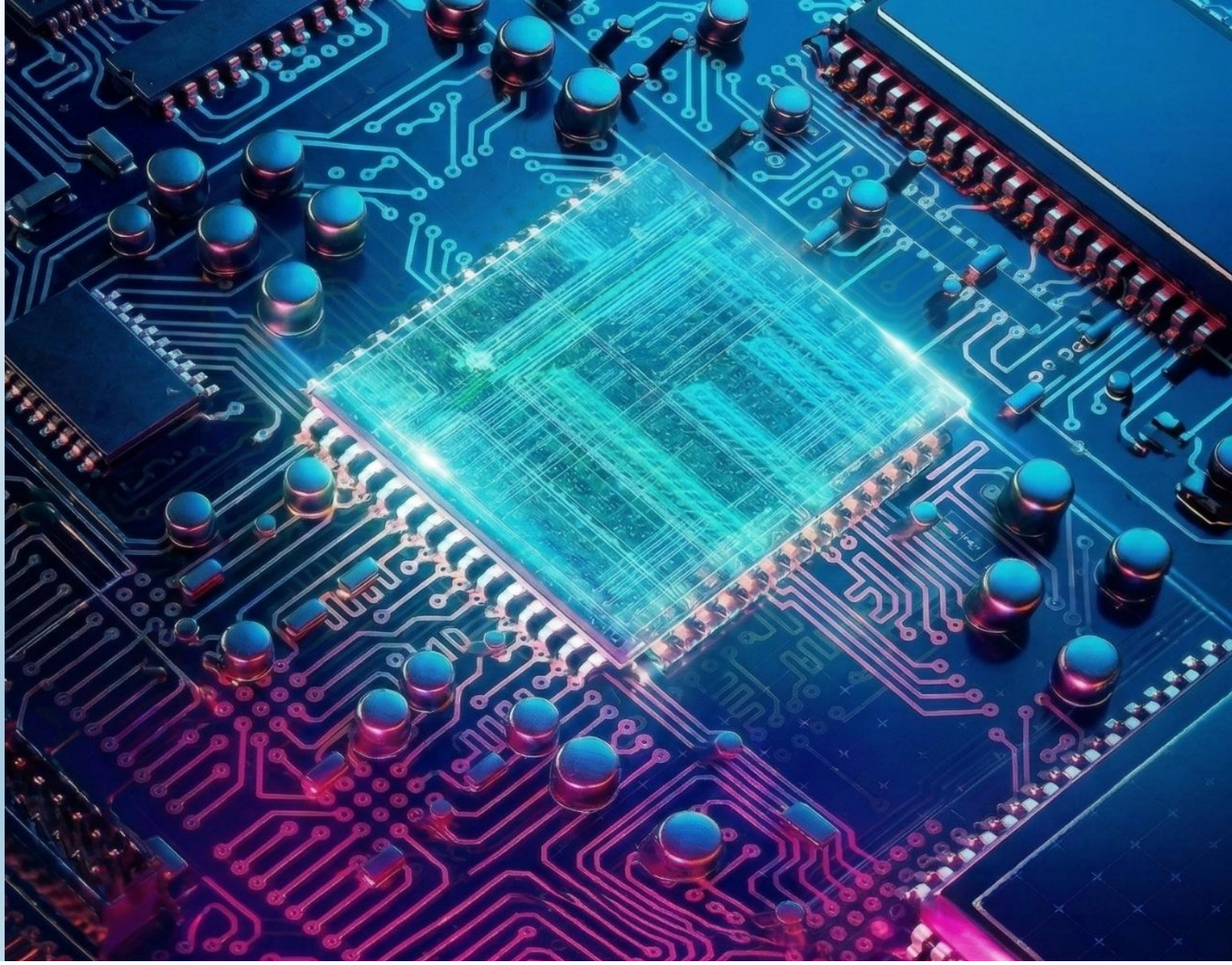


Digital Verification Extra Assignment 1

Name: Mohamed Torki

Alexandria University, Faculty of Engineering

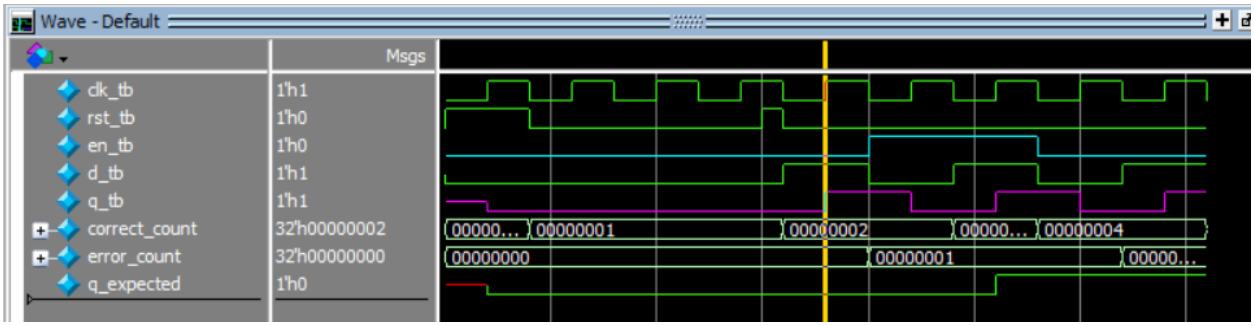
Electronics and Communications Department

Question 1:

Design issues:

q changes despite that enable isn't asserted

```
20: Error: For d_tb = 1 | en_tb = 0 | q_tb should equal 0 but is 1
32: Error: For d_tb = 0 | en_tb = 0 | q_tb should equal 1 but is 0
36: At end of test error count is 2 and correct count = 5
```



Assignments > A1 > Qextra > dff.v > dff

```
1  module dff(clk, rst, d, q, en);
2  parameter USE_EN = 1;
3  input clk, rst, d, en;
4  output reg q;
5
6  always @(posedge clk) begin
7     if (rst)
8        q <= 0;
9     else
10       if(USE_EN)
11          if (en)
12             q <= d; //missing else statment
13       else
14          q <= d;
15 end
16
17 endmodule
18
```

Faulty Design

Assignments > A1 > Qextra > dff.v > ...

```
1  module dff(clk, rst, d, q, en);
2  parameter USE_EN = 1;
3  input clk, rst, d, en;
4  output reg q;
5
6  always @(posedge clk) begin
7     if (rst)
8        q <= 0;
9     else
10       if(USE_EN)
11          if (en)
12             q <= d;
13          else
14             q <= q;
15       else
16          q <= d;
17 end
18
19 endmodule
20
```

Correction

```
76: At end of test error count is 0 and correct count = 17
```

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
dff_0	When the reset is asserted, the output q value must be low synchronously with positive clk edge	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
dff_1	exhaustive testing patterns where all combinations of inputs are tested	Directed during the simulation	-	A checker in the testbench to make sure the output is correct

First Testbench:

Assignments > A1 > Qextra > dff_t1_tb.sv > dff_t1_tb > DUT0

```
1   module dff_t1_tb;
2   bit clk_tb, rst_tb, en_tb;
3   logic d_tb ;
4   logic q_tb;
5
6   localparam USE_EN_tb = 1;
7
8   integer correct_count, error_count;
9
10  initial begin
11      clk_tb = 0;
12      forever
13          #2 clk_tb = ~clk_tb;
14  end
```

Declaring Signals, Instantiating DUT and Generating a clock

```
16  //reference model
17  logic q_expected;
18  always@(posedge clk_tb)begin
19      if(rst_tb)
20          q_expected = 0;
21      else if(en_tb)
22          q_expected = d_tb;
23      else
24          q_expected = q_expected;
25  end
26
27  dff #(.USE_EN(USE_EN_tb)) DUT0(
28      .clk(clk_tb),
29      .rst(rst_tb),
30      .en(en_tb),
31      .d(d_tb),
32      .q(q_tb)
33  );
34
```

Reference Model

```
35  initial begin
36      rst_tb = 0;
37      en_tb = 0;
38      d_tb = 0;
39      error_count = 0;
40      correct_count = 0;
41
42      //dff_0
43      assert_reset();
44      repeat(2)@(negedge clk_tb)
45      #3;
46      assert_reset();
47
48      //dff_1
49      for(int i = 0; i < 15; i++) begin
50          {en_tb,d_tb}++;
51          check_result();
52      end
53
54      $display("%t: At end of test error count is %0d and correct count = %0d",
55                  $time, error_count, correct_count);
56      $stop;
57  end
```

Verifying reset functionality

Testing all possible inputs

```
59  task check_result;
60      @(negedge clk_tb);
61      if(q_expected != q_tb) begin
62          error_count ++;
63          $display("%t: Error: For d_tb = %0d | en_tb = %0d | q_tb should equal %0d but is %0d",
64                  $time, d_tb, en_tb,q_expected,q_tb);
65      end
66      else
67          correct_count++;
68  endtask
69
70  task assert_reset;
71  rst_tb = 1;
72  check_result();
73  rst_tb = 0;
74  endtask
75
76
77  endmodule
```

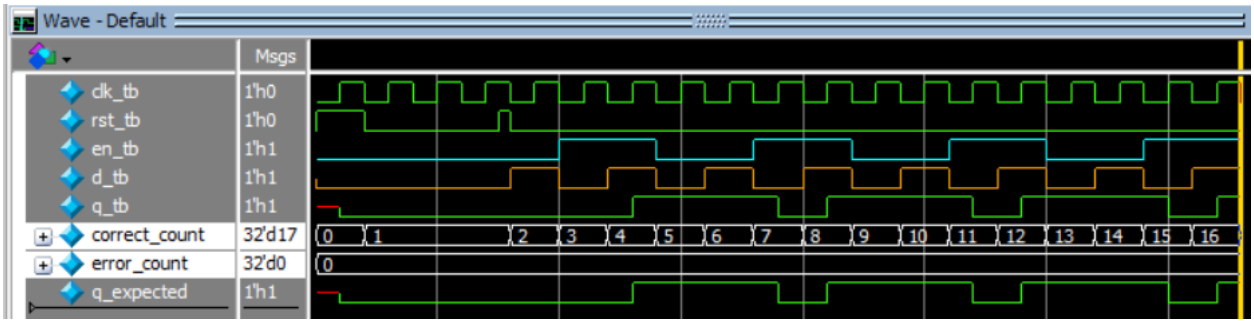
Check result and assert reset tasks

Do File:

Assignments > A1 > Qextra > run.do

```
1   vlib work
2   vlog dff.v dff_t1_tb.sv  +cover -covercells
3   vsim -voptargs=+acc work.dff_t1_tb -cover
4   add wave *
5   coverage save dff_t1_tb.ucdb -onexit
6   run -all
```

Waveform:



Second Testbench:

Assignments > A1 > Qextra > dff_t2_tb.sv > dff_t2_tb > DUT1

```
1   module dff_t2_tb;
2   bit clk_tb, rst_tb, en_tb;
3   logic d_tb ;
4   logic q_tb;
5
6   localparam USE_EN_tb = 0;
7
8   integer correct_count, error_count;
9
10  initial begin
11      clk_tb = 0;
12      forever
13          #2 clk_tb = ~clk_tb;
14  end
```

Declaring Signals, Instantiating DUT and Generating a clock

```
16  //reference model
17  logic q_expected;
18  always@(posedge clk_tb)begin
19      if(rst_tb)
20          q_expected <= 0;
21      else
22          q_expected <= d_tb;
23  end
24
25  dff #(.USE_EN(USE_EN_tb)) DUT1(
26      .clk(clk_tb),
27      .rst(rst_tb),
28      .en(en_tb),
29      .d(d_tb),
30      .q(q_tb)
31  );
32
```

Reference Model

```
33  initial begin
34      rst_tb = 0;
35      en_tb = 0;
36      d_tb = 0;
37      error_count = 0;
38      correct_count = 0;
39
40      //dff_0
41      assert_reset();
42      repeat(2)@(negedge clk_tb)
43      #3;
44      assert_reset();
45
46      //dff_1
47      for(int i = 0; i < 15; i++) begin
48          {en_tb,d_tb}++;
49          check_result();
50      end
51
52      $display("%t: At end of test error count is %0d and correct count = %0d",
53                  $time, error_count, correct_count);
54      $stop;
55  end
56
```

Verifying reset functionality

Testing all possible inputs

```
57  task check_result;
58      @(negedge clk_tb);
59      if(q_expected != q_tb) begin
60          error_count ++;
61          $display("%t: Error: For d_tb = %0d | en_tb = %0d | q_tb should equal %0d but is %0d",
62                  $time, d_tb, en_tb,q_expected,q_tb);
63      end
64      else
65          correct_count++;
66  endtask
67
68  task assert_reset;
69  rst_tb = 1;
70  check_result();
71  rst_tb = 0;
72  endtask
73
74  endmodule
```

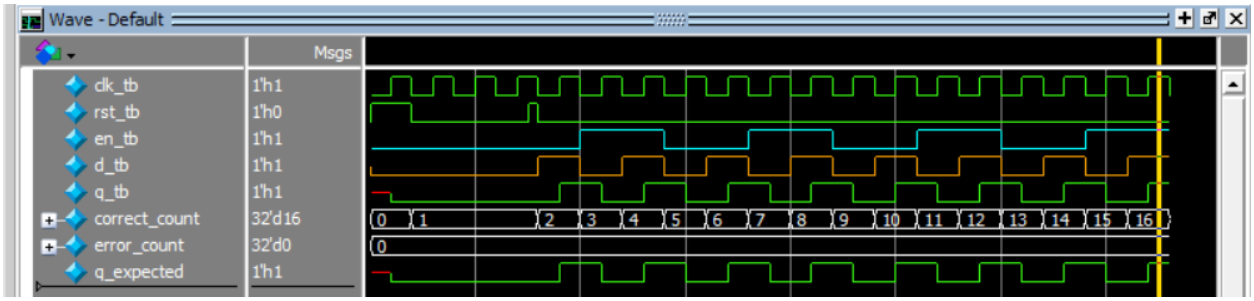
Check result and assert reset tasks

Do File:

Assignments > A1 > Qextra > run.do

```
1   vlib work
2   vlog dff.v dff_t2_tb.sv  +cover -covercells
3   vsim -voptargs=+acc work.dff_t2_tb -cover
4   add wave *
5   coverage save dff_t2_tb.ucdb -onexit
6   run -all
```

Waveform:



Questasim commands to merge reports:

```
QuestaSim> vcover merge dff_merged.ucdb dff_t1_tb.ucdb dff_t2_tb.ucdb -du dff
```

```
QuestaSim> vcover report dff_merged.ucdb -details -annotate -all -output coverage_rpt.txt
```

Coverage Report:

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         4         4         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\work.dff

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File dff.v
-----------------------------------IF Branch-----------------------------------
    7                                         34     Count coming in to IF
    7              1                           2        if (rst)
    11             1                           8             if (en)
    13             1                           8             else
    9              1                          16        else
Branch totals: 4 hits of 4 branches = 100.00%
```

Branch Coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       5         5         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\work.dff  --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File dff.v
    1                                                  module dff(clk, rst, d, q, en);
    2                                                  parameter USE_EN = 1;
    3                                                  input clk, rst, d, en;
    4                                                  output reg q;
    5
    6              1                          34     always @(posedge clk) begin
    7                                                     if (rst)
    8              1                           2           q <= 0;
    9                                                    else
    10                                                      if(USE_EN)
    11                                                         if (en)
    12             1                           8              q <= d;
    13                                                      else
    14             1                           8              q <= q;
    15                                                   else
    16             1                          16              q <= d;
```

Statement Coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         10        10         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\worked  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                           clk           2           2      100.00
                                             d           2           2      100.00
                                            en           2           2      100.00
                                             q           2           2      100.00
                                           rst           2           2      100.00

Total Node Count     =          5
Toggled Node Count   =          5
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (10 of 10 bins)


Total Coverage By Instance (filtered view): 100.00%
```

Toggle Coverage